JavaScript Foundations — Course Notes

Milestone 2 & Milestone 3

Milestone 2: JavaScript Foundations with a Problem-Solving Mindset

---

Running JavaScript in VS Code

JavaScript can run two ways: in the browser (linked via a <script> tag, or typed into the browser console) or on your machine using Node.js. For practice, save a file as script.js and run it from the terminal with node script.js, or link it to an HTML file and check the output with console.log() in the browser's Developer Tools console.

```
<script src="script.js"></script>

console.log("Hello, JavaScript!");
```

Math Needed Before Programming

You don't need advanced math to start coding, but you should be comfortable with basic arithmetic (+, -, *, /), remainders (%), order of operations (PEMDAS), negative numbers, and comparing numbers—these show up constantly in conditions, loops, and calculations.

What Is a Variable?

A variable is a named container that stores a value in memory so it can be reused, passed around, and updated later instead of retyping the value everywhere. Think of it as a

labeled box: the label is the variable name, and the box holds whatever value you place inside.

The 5 Things Needed to Declare a Variable

A complete, valid variable declaration always has five parts, in order:

1. A declaration keyword—let, const, or var
2. A variable name—e.g., totalPrice
3. An equals sign (=) — the assignment operator
4. A value — the data being stored
5. A semicolon (;)—ends the statement

Missing any one of these (especially the keyword or the value) is one of the most common beginner syntax errors.

```
let totalPrice = 250;
//  ①    ②      ③  ④  ⑤
// ① keyword ② name ③ = ④ value ⑤ ; (end of line)
```

Variable Data Types — Overview

JavaScript is a dynamically typed language: you don't declare a type, JavaScript figures it out from the value you assign, and a variable's type can even change if you reassign it. The primitive data types you'll use constantly at this stage are Number, String, and boolean—with undefined and null appearing naturally as well.

Number Data Type

Represents both whole numbers (integers) and decimals (floats) — JavaScript does not have separate int/float types like some languages. Numbers can be positive, negative, zero, or written in scientific notation for very large/small values. Dividing by 0 returns Infinity, and an invalid math operation returns NaN (Not a Number).

```
let age = 25;          // integer
let price = 19.99;     // decimal
let temp = -5;          // negative
let big = 2.5e6;        // scientific notation = 2,500,000
console.log(10 / 0);    // Infinity
console.log("abc" * 2); // NaN
```

String Data Type

Represents text, always wrapped in quotes — single ('...'), double ("..."), or backticks (template literals). Strings can be joined with the + operator (concatenation) and are zero-indexed, meaning str[0] is the first character.

```
let firstName = "Alice";
let lastName = 'Khan';
let fullName = firstName + " " + lastName; // "Alice Khan"
console.log(firstName[0]); // "A"
```

Boolean Data Type

Represents exactly two possible values: true or false. Booleans are the result of any comparison or logical check, and they drive every decision your program makes in conditionals and loops.

```
let isLoggedIn = true;
let hasPermission = false;
console.log(5 > 3); // true (a Boolean produced by comparison)
```

undefined and null

"Undefined" is JavaScript's automatic value for a variable that has been declared but not yet assigned anything. 'Null' is different—it's a value a developer deliberately assigns to represent 'intentionally empty.' ' Both are falsy, but they mean different things when debugging.

```
let x;
console.log(x); // undefined (declared, no value yet)

let y = null;
console.log(y); // null (intentionally empty)
```

Checking a Variable's Type — typeof

The typeof operator returns a string naming a value's data type. It's one of the most useful tools for debugging unexpected values or verifying user input.

```
console.log(typeof 25);        // "number"
console.log(typeof "hi");      // "string"
console.log(typeof true);      // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null);      // "object"  (a well-known JS quirk)
```

Declaring Multiple Variables

You can declare several variables of different types in the same program and mix declaration with immediate assignment or declare-then-assign-later.

```
let age = 25;          // Number
let name = "Alice";     // String
let isStudent = true;   // Boolean
let score;               // declared, no value = undefined
score = 90;              // assigned later
```

JavaScript Keywords

Keywords are reserved words that JavaScript uses for its own syntax—they can never be used as a variable, function, or parameter name because the language already gives them special meaning. Common ones you'll meet early on include: let, const, var, function, return, if, else, for, while, break, continue, true, false, null, undefined, new, this, class, typeof, in, and of.

```
// ❌ Invalid — 'let' and 'return' are reserved keywords
let let = 5;
let return = 10;

// ☑ Valid — descriptive, non-reserved names
let totalScore = 5;
let returnValue = 10;
```

Variable Naming Rules (Must-Follow)

JavaScript enforces these rules strictly—breaking any of them causes a SyntaxError:

• Must start with a letter, underscore (_), or dollar sign ($)—never a digit

• Can contain letters, digits, underscores, and dollar signs after the first character

• Cannot contain spaces or hyphens

• Cannot be a reserved keyword

• Is case-sensitive (age and Age are two different variables)

```
// ❌ Invalid names
let 2names = "x";      // starts with a digit
let first-name = "x";  // hyphen not allowed
let user name = "x";   // space not allowed

// ☑ Valid names
let _score = 10;
let $price = 99;
let userName2 = "Alice";
```

Naming Conventions (Best Practice)

Beyond what JavaScript requires, good developers follow shared style conventions so code reads consistently across a team:

• camelCase for variables and functions — firstName, calculateTotal

• UPPER_SNAKE_CASE for values that never change — MAX_USERS, PI

• Names should be descriptive, not abbreviated—use totalPrice, not tp or x

• Boolean variables often start with is/has/can—isActive, hasAccess

```
let firstName = "Alice";      // camelCase ☑
const MAX_ATTEMPTS = 3;        // constant style ☑
```

```
let isEligible = true;          // boolean naming ☑

let fn = "Alice";               // ❌ works, but unclear/abbreviated
```

JavaScript Numbers — Fundamentals

All numbers in JavaScript—whole numbers and decimals—share a single Number type internally stored as a 64-bit floating-point value. This causes a well-known quirk: some decimal arithmetic isn't perfectly precise, because computers store decimals in binary.

```
console.log(0.1 + 0.2); // 0.30000000000000004 (not exactly 0.3)
console.log(Number.isInteger(10)); // true
console.log(Number("42"));         // 42 (string converted to number)
```

Introduction to Arithmetic Operators

The core arithmetic operators are + (add), - (subtract), * (multiply), / (divide), % (modulus — returns the remainder of a division), and ** (exponent — raises to a power). Order of operations follows standard math rules: parentheses first, then * / %, then + -.

```
let sum = 10 + 5;       // 15
let remainder = 10 % 3;  // 1
let power = 2 ** 3;      // 8 (2 to the power of 3)
let result = (2 + 3) * 4; // 20 (parentheses first)
```

(Advanced) Mathematical Operation Shorthand

Compound assignment operators let you update a variable using its own current value in fewer keystrokes — very common in loops and counters.

```
let count = 5;
count += 1;   // same as count = count + 1  → 6
count -= 1;   // same as count = count - 1  → 5
count *= 2;   // same as count = count * 2  → 10
count /= 5;   // same as count = count / 5  → 2
count++;      // increment by 1 → 3
count--;      // decrement by 1 → 2
```

Compare Variables & the Comparison Operators (Full Reference)

A comparison operator compares two values and always evaluates to a Boolean (true or false). This is the foundation every conditional is built on.

== Equal to (loose—converts type before comparing)
=== Strictly equal to (value AND type must match)
!= Not equal to (loose)

!== Strictly not equal to (value AND type)

> Greater than

< Less than

>= Greater than or equal to

<= Less than or equal to

```
console.log(5 > 3);      // true
console.log(5 < 3);      // false
console.log(5 >= 5);     // true
console.log(5 == "5");   // true  (loose: converts "5" to 5)
console.log(5 === "5");  // false (strict: number vs string)
console.log(5 !== "5");  // true
console.log(5 != "5");   // false
```

Why === Is Preferred Over ==

== silently converts types before comparing, which can produce results that look correct but hide bugs (e.g. 0 == "" is true, and "" == false is true). === never converts, so what you see is exactly what's compared. Professional codebases almost always use === and !== by default, reserving == only for the rare, intentional case.

Introduction to Conditionals

Conditionals let your program make decisions and run different code depending on whether a condition is true or false. Without conditionals, a program can only ever run the same fixed sequence of instructions — conditionals are what make software 'smart.'

if / else Statement

An if statement runs a block only when its condition is true; the optional else block runs when it's false. The condition inside if() must evaluate to a Boolean (or something JavaScript can convert to one).

```
let age = 18;
if (age >= 18) {
  console.log("You can vote.");
} else {
  console.log("Not eligible yet.");
}
```

Conditional Branching

Branching means the program takes one path or another based on a condition, rather than always running the same lines top to bottom—like a fork in the road that JavaScript decides between at runtime, every single time that line runs.

Multiple Conditions—Logical Operators (&& and ||)

&& (AND) requires every condition joined by it to be true before the whole expression is true. || (OR) requires only one of the joined conditions to be true. Combine multiple checks into a single condition instead of nesting many separate if statements.

```
let age = 20, hasID = true;
if (age >= 18 && hasID) {
  console.log("Entry allowed");   // both must be true
}

let isMember = false, hasCoupon = true;
if (isMember || hasCoupon) {
  console.log("Discount applied"); // only one needs to be true
}
```

Multi-Level if…else if…else

Chain else if to test several conditions in order; JavaScript checks them top to bottom and stops at the first one that's true, skipping the rest—so order matters, especially with overlapping ranges.

```
let score = 75;
if (score >= 90) console.log("A");
else if (score >= 75) console.log("B");
else if (score >= 60) console.log("C");
else console.log("F");
```

(Optional) Nested if…else Condition

An if statement placed inside another if block, used when a decision genuinely depends on more than one layer of condition. Nesting more than 2–3 levels deep usually means the logic could be simplified with && or early returns.

```
let age = 20, hasTicket = true;
if (age >= 18) {
  if (hasTicket) {
    console.log("Welcome in!");
  } else {
    console.log("Please buy a ticket.");
  }
} else {
  console.log("Must be 18+.");
}
```

(Advanced) If-Else Shorthand — Ternary Operator

The ternary operator (condition ? valueIfTrue : valueIfFalse) is a compact one-line shorthand for a simple if/else that returns a value, ideal for quick assignments.

```
let age = 16;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status); // "Minor"
```

(Advanced) Logical NOT Operator

The "!" operator flips a Boolean value—true becomes false and false becomes true. It's handy for checking that something is NOT the case, and it also converts a truthy/falsy value into its opposite Boolean.

```
let isLoggedIn = false;
if (!isLoggedIn) {
  console.log("Please log in.");
}
console.log(!true);  // false
console.log(!"");     // true ("" is falsy, so !"" is true)
```

What Is a Loop?

A loop repeats a block of code multiple times without you having to write it out repeatedly — essential whenever you need to process a range of numbers, repeat an action a set number of times, or work through every item in a collection.

Introduction to the While Loop

A while loop keeps running as long as its condition stays true. It checks the condition before every pass, so if the condition is false from the start, the loop body never runs at all. You must update the condition variable inside the loop, or it will run forever (an infinite loop).

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;   // without this line, the loop never ends
}
```

Problem-Solving with While Loop

While loops are the natural fit when the number of repetitions isn't known ahead of time—for example, repeating an action until a user enters valid input, or until a running total crosses a threshold.

Introduction to the For Loop

A for loop packs initialization, condition, and update into one line — the most common loop for a known number of repetitions. Its three parts run in this order: initializer once, then condition check → body → update, repeating until the condition is false.

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
// i = 0: init once
// i < 5: condition checked before every pass
// i++  : update after every pass
```

Problem-Solving with For Loop

for loops are the natural fit when you already know how many times to repeat—looping a fixed number of times, or iterating through every index of an array.

Different Ways to Use a Loop

Loops can count up or down, skip by more than one, start from a non-zero index, or iterate directly over array/string elements—the structure is flexible as long as the condition eventually becomes false.

```
for (let i = 10; i > 0; i--) console.log(i);      // counts down
for (let i = 0; i < 10; i += 2) console.log(i);   // skips by 2
```

When to Use break and continue

break exits the loop entirely and jumps to the code right after it—use it when you've found what you're looking for and no longer need to keep looping. continue skips only the rest of the current iteration and moves straight to the next one—use it to skip specific values without stopping the whole loop.

```
for (let i = 0; i < 10; i++) {
  if (i === 5) break;        // stop the loop completely at 5
  if (i % 2 === 0) continue;  // skip even numbers, keep looping
  console.log(i);             // logs 1, 3
}
```

(Optional) Introduction to Do While Loop

A do...while loop runs its block at least once before checking the condition, unlike a regular while loop that checks first—useful when an action must happen at least one time regardless of the condition (like showing a menu once before checking whether to repeat it).

```
let i = 0;
do {
  console.log(i);
  i++;
```

```
} while (i < 3);

// Even if the condition starts false, the body still runs once:
let j = 10;
do {
  console.log("Runs once");
} while (j < 5);
```

Difference Between Types of Loops — Summary

Use for when you know how many times to repeat (a fixed range or array length). Use while when repetition depends on a changing condition whose end point isn't known in advance. Use do…while when the loop body absolutely must run at least once before the condition is ever checked.

Store More Than One Value Using an Array

An array stores an ordered list of values in a single variable, so you don't need a separate variable for every item. Arrays can hold any data type, even a mix of types, and are written with square brackets.

```
let fruits = ["apple", "banana", "mango"];
let mixed = [1, "two", true, null];
```

Array Length, Index, Get and Set by Index

Array items are indexed starting at 0, so the first element is arr[0] and the last is arr[arr.length - 1]. Use .length to get the total number of elements, and bracket notation to both read and overwrite a value at a specific index.

```
console.log(fruits[0]);       // "apple" (get)
console.log(fruits.length);   // 3
fruits[1] = "orange";          // set / overwrite index 1
console.log(fruits[fruits.length - 1]); // last element
```

Add/Remove Elements — push, pop, shift, unshift

push() adds one or more elements to the end and returns the new length. pop() removes the last element and returns it. unshift() adds to the beginning (shifting every other index up by one). shift() removes the first element (shifting every other index down by one). All four mutate the original array.

```
let arr = [2, 3, 4];
arr.push(5);      // [2,3,4,5]
arr.pop();         // [2,3,4]   (removed 5)
arr.unshift(1);    // [1,2,3,4]
arr.shift();       // [2,3,4]   (removed 1)
```

Array Methods Reference — Full List with Examples

Beyond push/pop/shift/unshift, JavaScript arrays come with a rich set of built-in methods. Mutating methods change the original array; non-mutating methods return a new array/value and leave the original untouched.

```
// splice(start, deleteCount, ...items) — MUTATES: add/remove anywhere
let a = [1,2,3,4,5];
a.splice(1, 2, "x", "y"); // removes 2 items from index 1, inserts x,y
console.log(a); // [1,"x","y",4,5]

// slice(start, end) — NON-mutating: copies a portion
let b = [1,2,3,4,5];
console.log(b.slice(1,3)); // [2,3] — b unchanged

// concat() — NON-mutating: merges arrays
console.log([1,2].concat([3,4])); // [1,2,3,4]

// join(separator) — turns array into a string
console.log([1,2,3].join("-")); // "1-2-3"

// indexOf() / lastIndexOf() — find a value's index
console.log([5,6,7,6].indexOf(6));     // 1 (first match)
console.log([5,6,7,6].lastIndexOf(6)); // 3 (last match)

// includes() — true/false, does the array contain a value?
console.log([1,2,3].includes(2)); // true

// Array.isArray() — checks whether a value is an array
console.log(Array.isArray([1,2])); // true

// flat() — flattens nested arrays
console.log([1,[2,3],[4,[5]]].flat(2)); // [1,2,3,4,5]

// fill() — fills all/part of an array with a static value
console.log([1,2,3,4].fill(0, 1, 3)); // [1,0,0,4]

// Array.from() — builds an array from an iterable or array-like value
console.log(Array.from("abc")); // ["a","b","c"]
```

Array Traversal Using for and while Loop

Traversal means visiting every element in an array, usually with a loop that runs from index 0 to length - 1, reading (or processing) each value along the way.

```
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

```
let i = 0;
while (i < fruits.length) {
  console.log(fruits[i]);
  i++;
}
```

Reversing an Array With/Without reverse()

The built-in .reverse() method flips the array in place (mutates the original). You can also reverse manually by swapping elements from both ends toward the middle, without a built-in method—good practice for understanding how the method works internally.

```
let nums = [1, 2, 3];
nums.reverse(); // [3, 2, 1] — mutates original

// Manual version (no built-in method):
function reverseArray(arr) {
  let result = [];
  for (let i = arr.length - 1; i >= 0; i--) {
    result.push(arr[i]);
  }
  return result;
}
```

Sort an Array — Problems and Solutions

The .sort() method sorts as strings by default, which breaks numeric order (e.g. 10 comes before 2)—pass a compare function to sort numbers correctly. (a, b) => a - b sorts ascending; (a, b) => b - a sorts descending.

```
let nums = [10, 2, 33, 4];
console.log(nums.sort());               // [10,2,33,4] ❌ wrong (string sort)
console.log(nums.sort((a, b) => a - b)); // [2,4,10,33] ☑ ascending
console.log(nums.sort((a, b) => b - a)); // [33,10,4,2] ☑ descending
```

Introduction to Strings vs. Arrays

A string is a sequence of characters, similar to an array of characters—both have a .length and can be accessed by index. The key difference: strings are immutable (you cannot change a single character in place; you must create a new string), while array elements can be reassigned directly.

String Methods Reference — Full List with Examples

Strings come with a large set of built-in methods for reading, searching, and transforming text. None of them change the original string — they always return a new string or value.

```
let str = "  Hello World  ";

// case & whitespace
str.toLowerCase();     // "  hello world  "
str.toUpperCase();     // "  HELLO WORLD  "
str.trim();             // "Hello World" (removes both ends)
str.trimStart();        // removes leading whitespace only
str.trimEnd();           // removes trailing whitespace only

// searching
str.includes("World");  // true
str.startsWith("  He"); // true
str.endsWith("ld  ");    // true
str.indexOf("o");        // 6 (first match)
str.lastIndexOf("o");    // 9 (last match)

// extracting
str.charAt(2);           // "H"
str.at(-1);               // last character (supports negative index)
str.slice(2, 7);          // "Hello"
str.substring(2, 7);      // "Hello" (similar to slice, no negative index)

// transforming
str.replace("World", "JS");     // replaces first match
str.replaceAll("o", "0");        // replaces every match
str.split(" ");                   // ["","","Hello","World","",""]
str.concat("!");                  // joins strings
"ab".repeat(3);                    // "abababab"
"5".padStart(3, "0");              // "005"
"5".padEnd(3, "0");                // "500"
```

String Comparison — Lowercase, Uppercase and Trim

Because strings are case-sensitive ("Apple" !== "apple"), always normalize both sides with toLowerCase() (or toUpperCase()) before comparing user input. trim() removes accidental leading/trailing whitespace that often causes 'why doesn't this match?!' bugs.

```
let name = "  Alice  ";
console.log(name.trim().toLowerCase() === "alice"); // true
```

String Slice, Join, Concatenate and Includes

slice() extracts part of a string by start/end index, concat() (or the '+' operator) joins strings together, and includes() checks whether a substring exists anywhere inside the string.

```
let str = "Hello World";
console.log(str.slice(0, 5));        // "Hello"
console.log(str.concat("!"));         // "Hello World!"
console.log(str.includes("World"));   // true
```

Reverse a String in Three Different Ways

Strings have no built-in .reverse() (that method belongs to arrays), so you either (1) convert to an array, reverse, and join back; (2) loop backward manually, building a new string character by character; or (3) use recursion.

```
// Method 1: split → reverse → join
let s1 = "hello".split("").reverse().join(""); // "olleh"

// Method 2: manual loop
function reverseLoop(str) {
  let result = "";
  for (let i = str.length - 1; i >= 0; i--) result += str[i];
  return result;
}

// Method 3: recursion
function reverseRecursive(str) {
  if (str === "") return "";
  return reverseRecursive(str.slice(1)) + str[0];
}
```

Introduction to Objects — Properties and Values

An object stores data as key-value pairs (properties), rather than an ordered list—ideal for representing a single 'thing' with several named attributes. Each key maps to a value, which can be any data type, including another object or a function.

```
let person = { name: "Alice", age: 25, isStudent: true };
```

Multiple Ways to Get & Set Object Properties

Use dot notation for known, fixed property names—it's shorter and more readable. Use bracket notation when the property name is dynamic (stored in a variable) or contains characters dot notation can't handle (like spaces).

```
console.log(person.name);       // dot notation
console.log(person["age"]);     // bracket notation
person.age = 26;                 // update via dot
```

```
let key = "name";
console.log(person[key]);        // dynamic bracket access → "Alice"
```

Object Methods Reference — Full List with Examples

The Object built-in provides several static methods for inspecting and transforming objects as a whole.

```
let obj = { a: 1, b: 2, c: 3 };

Object.keys(obj);       // ["a","b","c"]
Object.values(obj);     // [1,2,3]
Object.entries(obj);    // [["a",1],["b",2],["c",3]]

// assign() — merges objects into a new/target object
let merged = Object.assign({}, obj, { d: 4 }); // {a:1,b:2,c:3,d:4}

// hasOwnProperty() — checks if a key belongs directly to the object
console.log(obj.hasOwnProperty("a")); // true

// freeze() / seal() — restrict modification
Object.freeze(obj);   // fully locked: no add/remove/change
Object.seal(obj);     // can edit existing values, but not add/remove keys

// fromEntries() — builds an object back from an entries array
let entries = [["x",1],["y",2]];
console.log(Object.fromEntries(entries)); // {x:1, y:2}
```

Keys, Values, Nested Objects and delete

Object.keys() and Object.values() return arrays of an object's property names and values. Objects can be nested inside other objects to represent more complex data, accessed by chaining dot/bracket notation. delete removes a property entirely from the object.

```
let user = {
  name: "Alice",
  address: { city: "Dhaka", zip: "1207" }
};
console.log(user.address.city); // "Dhaka" (nested access)
delete user.address.zip;         // removes zip from the nested object
```

Loop an Object and Ways to Declare an Object

Use a for...in loop to iterate over an object's keys, since a regular for loop only works on indexed collections like arrays. Objects can also be created with the shorthand { } literal (most common) or with the new Object() constructor.

```
for (let key in person) {
  console.log(key, person[key]);
}
```

```
// Two ways to create an object:
let obj1 = { name: "Bob" };       // object literal (preferred)
let obj2 = new Object();           // constructor (rarely used)
obj2.name = "Bob";
```

What Are Functions and Function Syntax

A function is a reusable, named block of code that performs a specific task. Defining it once and calling it by name avoids repeating the same logic throughout your program, and makes code easier to test and maintain.

```
function greet() {
  console.log("Hello!");
}
greet(); // call/invoke the function
```

Function Parameter, Handle Multiple Parameters

Parameters are placeholders listed in the function definition; a function can accept multiple parameters separated by commas, each becoming a local variable available inside the function body.

```
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // 5
```

How Function Works — Argument vs Parameter

Parameters exist in the function's definition (the placeholder names). Arguments are the real values supplied at call time. If fewer arguments are passed than parameters expect, the missing ones become undefined inside the function.

```
function multiply(x, y) { // x, y = parameters
  return x * y;
}
multiply(4, 5); // 4, 5 = arguments → returns 20
multiply(4);     // y is undefined → returns NaN
```

Function Return & Set Return Value to a Variable

return sends a value back out of the function and immediately ends its execution—any code after return inside that function never runs. Without an explicit return, a function outputs undefined. The returned value can be stored in a variable for later use.

```
function square(n) {
  return n * n;
  console.log("never runs"); // unreachable
}
let result = square(4); // result = 16
```

Recap and Conditional Return of Odd and Even

A function can return different values depending on a condition inside it, combining functions with conditionals—a very common real-world pattern.

```
function checkEvenOdd(n) {
  if (n % 2 === 0) return "Even";
  return "Odd";
}
```

Different Types of Parameters for a Function

Parameters can be required, given default fallback values, or collect an unknown number of arguments (rest parameters)—the default-value and rest-parameter syntax specifically is covered in full with ES6.

Sum of All Numbers in an Array Using Function

A common function pattern: loop through an array, accumulating a running total in a variable initialized before the loop, then return the total once the loop finishes.

```
function sumArray(arr) {
  let total = 0;
  for (let i = 0; i < arr.length; i++) {
    total += arr[i];
  }
  return total;
}
console.log(sumArray([1,2,3,4])); // 10
```

Return All the Even Numbers of an Array

Combine a loop, a conditional, and an empty result array to filter values manually—this is exactly what the built-in .filter() method does internally.

```
function getEvens(arr) {
  let evens = [];
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] % 2 === 0) evens.push(arr[i]);
  }
  return evens;
}
console.log(getEvens([1,2,3,4,5,6])); // [2,4,6]
```

Function Summary and Practice

Functions are the single most important tool for organizing code. Practice writing small, single-purpose functions that take clear input (parameters) and always return a predictable output—this discipline pays off enormously as programs grow.

The Problem-Solving Mindset

Before coding, restate the problem in your own words, identify the inputs and expected output, and sketch the steps in plain language or pseudocode. This habit — understand, plan, then code — prevents wasted effort and makes debugging dramatically faster.

Number Problems—Even/Odd Check & Sum of a Range

Use % 2 to check even/odd (remainder 0 means even) and a loop with a running total to sum every number in a range from start to end.

```
function isEven(n) { return n % 2 === 0; }

function sumRange(start, end) {
  let total = 0;
  for (let i = start; i <= end; i++) total += i;
  return total;
}
console.log(sumRange(1, 5)); // 15
```

Number Problems — Factorial & FizzBuzz

Factorial (n!) multiplies a number by every positive integer below it down to 1. FizzBuzz prints 'Fizz' for multiples of 3, 'Buzz' for multiples of 5, and 'FizzBuzz' for numbers that are multiples of both — a classic loop + conditional exercise used in almost every beginner course.

```
function factorial(n) {
  let result = 1;
  for (let i = 2; i <= n; i++) result *= i;
  return result;
}

for (let i = 1; i <= 15; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}
```

String Problems—Reverse a String & Count Vowels

Counting vowels means looping through each character of a string and incrementing a counter whenever the character matches a vowel.

```
function countVowels(str) {
  let count = 0;
  for (let ch of str.toLowerCase()) {
    if ("aeiou".includes(ch)) count++;
  }
  return count;
}
console.log(countVowels("Hello World")); // 3
```

String Problems — Palindrome Check & Count Words

A palindrome reads the same forward and backward—compare the (cleaned, lowercased) string to its own reversed version. Counting words means splitting the string on spaces and counting the resulting array's length.

```
function isPalindrome(str) {
  let clean = str.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === clean.split("").reverse().join("");
}
console.log(isPalindrome("Was it a car or a cat I saw")); // true

function countWords(str) {
  return str.trim().split(/\s+/).length;
}
```

Array Problems — Find the Largest & Smallest Value

Track a running max/min variable, initialized to the first element, and update it whenever a bigger (or smaller) value is found while looping through the rest.

```
function findMax(arr) {
  let max = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > max) max = arr[i];
  }
  return max;
}
```

Array Problems — Sum/Average & Filter by Condition

Average is the sum divided by the count of elements. Filtering by condition means building a new empty array and pushing into it only the elements that pass a test—manually, before .filter() is introduced.

```
function average(arr) {
  let total = 0;
  for (let n of arr) total += n;
  return total / arr.length;
}

function filterAbove(arr, limit) {
  let result = [];
  for (let n of arr)

      {

      if (n > limit)

      result.push(n);

      }
  return result;
}
```

Object Problems — Loop an Object & Find a Value

Combine for…in with a conditional to search an object's properties for a specific value or key that matches some criteria.

```
function findKeyByValue(obj, target) {
  for (let key in obj) {
    if (obj[key] === target) return key;
  }
  return null;
}
```

Putting It Together — Function + Loop + Condition

Real problems combine all three building blocks: a function to package the logic, a loop to process a collection, and a condition to make decisions along the way. Practicing this combination is the single best way to build problem-solving confidence.

Bug Basics — What Is a Bug?

A bug is any unexpected or incorrect behavior in a program — code that runs but produces the wrong result, or code that doesn't run at all. Bugs are a completely normal part of programming; professional developers spend a significant portion of their time debugging, not just writing new code.

Types of Errors — Full Breakdown

JavaScript errors generally fall into four categories, each with a distinct cause and a distinct fix strategy:

1. Syntax Error — the code breaks JavaScript's grammar rules (missing bracket, missing comma, typo in a keyword). The code won't run at all.

2. Reference Error — the code tries to use a variable or function that doesn't exist / hasn't been declared in the current scope.

3. Type Error — the code tries to perform an operation on a value of the wrong type (e.g. calling something that isn't a function, or reading a property of undefined).

4. Logic Error — the code runs without crashing, but produces the wrong output because the logic itself is flawed. This is the hardest type to catch, since JavaScript gives no error message at all.

```
// 1. Syntax Error
if (true {          // ❌ missing closing parenthesis
  console.log("hi");
}

// 2. Reference Error
console.log(total);  // ❌ 'total' was never declared

// 3. Type Error
let num = 5;
num();                // ❌ num is not a function

// 4. Logic Error (no crash, wrong result)
function average(a, b) {
  return a + b / 2;    // ❌ should be (a + b) / 2
}
```

Reading an Error Message (Anatomy)

A console error message tells you three critical things: the error type (e.g. TypeError), a short description of what went wrong, and a stack trace showing the file and line number where it happened, plus the chain of function calls that led there. Always read the error type and line number first — that's usually enough to locate the bug.

```
// Example console output:
// Uncaught TypeError: Cannot read properties of undefined (reading 'name')
//   at getUserName (app.js:12)
//   at main (app.js:20)
// → Error type: TypeError
// → Cause: reading 'name' on something undefined
// → Location: line 12, inside getUserName()
```

console.log() and Other Console Methods

console.log() prints any value so you can inspect it mid-execution — the single most-used debugging tool. Related methods give more targeted output: console.error() flags something as an error (shown in red), console.warn() flags a warning (yellow), console.table() prints arrays/objects as a readable table, and console.assert() only logs if a given condition is false.

```
console.log("value:", total);
console.error("Something went wrong!");
console.warn("This is deprecated");
console.table([{ name: "Alice", age: 25 }, { name: "Bob", age: 30 }]);
console.assert(total > 0, "total should be positive");
```

Debugging a Broken Conditional

Common conditional bugs: using = (assignment) instead of === (comparison), mismatched brackets, an inverted condition (missing !), or comparing values of different types. Print the condition's value right before the if statement to confirm exactly what it evaluates to.

```
let age = 20;
if (age = 18) { ... }   // ❌ assigns 18 to age, always truthy
if (age === 18) { ... } // ☑ actually compares

console.log("condition:", age === 18); // debug the condition directly
```

Debugging a Broken Loop

Common loop bugs: an off-by-one error in the condition (<= instead of <, or vice versa), forgetting to update the counter (infinite loop), or using the wrong comparison direction when counting down.

Debugging a Broken Array Problem

Common array bugs: looping one index too far (arr.length vs arr.length - 1, causing undefined), mutating the array while iterating over it (which shifts indexes mid-loop), or comparing array indexes incorrectly.

Debugging a Broken Function

Common function bugs: forgetting to return a value (function silently returns undefined), shadowing a parameter name with a variable of the same name inside the function, or calling the function with arguments in the wrong order.

Using the VS Code Debugger

VS Code lets you set breakpoints by clicking next to a line number, then run the file in Debug mode (Run → Start Debugging) to pause execution exactly there. While paused you can inspect every variable's current value, step through the code line by line (Step Over / Step Into / Step Out), and watch the Call Stack panel to see exactly how execution got there — far more powerful than scattering console.log() everywhere.

try…catch — Handling Errors Gracefully

try…catch lets your program catch a runtime error instead of crashing completely, so you can log it, show a friendly message, or recover. finally runs regardless of whether an error occurred.

```
try {
  let result = riskyOperation();
  console.log(result);
} catch (error) {
  console.error("Something failed:", error.message);
} finally {
  console.log("This always runs.");
}
```

The Full Debugging Process (Step-by-Step)

A repeatable process that works on almost any bug:

1. Reproduce — trigger the bug consistently so you can observe it
2. Read — read the full error message and stack trace carefully
3. Isolate — narrow down which line/function is responsible (comment out code, add logs, or use breakpoints)
4. Hypothesize — form a specific guess about the cause
5. Test — check the hypothesis with a log statement or breakpoint
6. Fix — make the smallest change that resolves the actual cause
7. Verify — re-run and confirm the fix works, and that nothing else broke

Milestone 3: Empowering JavaScript with ES6

ES6 Intro — var vs let vs const (Full Comparison)

ES6 (ECMAScript 2015) introduced let and const as modern replacements for var, fixing several of var's long-standing problems. The differences matter a lot in real code:

Scope: var is function-scoped (visible throughout the whole function it's declared in, ignoring blocks like if/for). let and const are block-scoped (only visible inside the nearest { } they're declared in).

Redeclaration: var can be redeclared with the same name in the same scope without error. let and const throw a SyntaxError if redeclared in the same scope.

Reassignment: var and let can both be reassigned. const cannot be reassigned after its initial value is set.

Hoisting: all three are hoisted (JavaScript is aware of the declaration before that line runs), but var is initialized as undefined and usable early, while let/const remain in a 'temporal dead zone' — accessing them before their declaration line throws a ReferenceError.

Rule of thumb: use const by default, switch to let only when the variable's value genuinely needs to change, and avoid var entirely in modern code.

```
// Scope difference
if (true) {
  var x = 1;
  let y = 2;
}
console.log(x); // 1  (var leaked out of the block)
console.log(y); // ❌ ReferenceError (let stayed inside the block)

// Redeclaration
var a = 1;
var a = 2;      // ☑ allowed, no error
let b = 1;
let b = 2;      // ❌ SyntaxError: 'b' has already been declared

// Reassignment
let score = 10;
```

```
score = 20;       // ☑ allowed
const PI = 3.14;
PI = 3.14159;     // ❌ TypeError: Assignment to constant variable

// Temporal Dead Zone
console.log(city); // ❌ ReferenceError (used before declaration)
let city = "Dhaka";
```

const With Objects and Arrays — A Common Misunderstanding

const prevents reassigning the variable itself, but it does NOT make the value inside it immutable. If a const holds an object or array, you can still change its contents (add/remove/edit) — you just can't point the variable at a completely different object or array.

```
const user = { name: "Alice" };
user.name = "Bob";      // ☑ allowed — editing a property, not reassigning
user.age = 25;            // ☑ allowed — adding a new property
user = { name: "Eve" };   // ❌ TypeError — reassigning the const itself

const nums = [1, 2, 3];
nums.push(4);              // ☑ allowed — mutating the array's contents
nums = [9, 9, 9];           // ❌ TypeError
```

Default Parameters for Not-Provided Values

A function parameter can have a default value that's used automatically if no argument (or undefined) is passed for it — cleaner than manually checking inside the function body.

```
function greet(name = "Guest") {
  console.log(`Hello, ${name}`);
}
greet();          // "Hello, Guest" (default used)
greet("Alice");   // "Hello, Alice"
```

Template Strings — Multiline and Dynamic Strings

Template literals (backticks) let you embed variables and expressions directly inside a string with ${...}, and write multiline strings without manual concatenation or \n.

```
let name = "Alice", age = 25;
console.log(`Hello, ${name}! You are ${age} years old.`);

console.log(`Line one
Line two
Line three`); // multiline, no \n needed
```

Arrow Functions — Syntax, Params, Return

Arrow functions are a shorter syntax for writing functions. For a single expression, the return keyword and braces can be dropped entirely (implicit return). Parentheses around a single parameter are optional; multiple parameters require them.

```
// Regular function
function add(a, b) { return a + b; }

// Arrow function — equivalent
const add2 = (a, b) => a + b;

// Single parameter — parentheses optional
const square = n => n * n;

// No parameters — parentheses required
const sayHi = () => console.log("Hi!");

// Multi-line body needs explicit braces + return
const divide = (a, b) => {
  if (b === 0) return "Cannot divide by zero";
  return a / b;
};
```

Spread Operator, Array Max, Copy Arrays

The spread operator (...) expands an array or object into individual elements — useful for copying arrays/objects, merging them, or passing array items as separate function arguments (like Math.max, which doesn't accept an array directly).

```
let nums = [3, 1, 4, 1, 5];
let max = Math.max(...nums);   // spreads array into separate arguments
let copy = [...nums];           // shallow copy, not the same reference
let merged = [...nums, 9, 9];   // merges + adds extra values
```

(Advanced) Object and Array Destructuring

Destructuring unpacks values from arrays or properties from objects into individual variables in one line, instead of accessing them one by one.

```
// Object destructuring
const { name, age } = { name: "Alice", age: 25 };
console.log(name, age); // Alice 25

// Array destructuring
const [first, second] = [10, 20];

// With renaming and default values
const { name: userName = "Guest" } = {};
console.log(userName); // "Guest"
```

Keys, Values, Entries, delete, seal & freeze

Object.entries() returns an array of [key, value] pairs, useful for looping with for...of. Object.seal() prevents adding/removing properties (existing ones can still be changed); Object.freeze() locks the object completely, blocking any addition, removal, or change.

```
let obj = { a: 1, b: 2 };
console.log(Object.entries(obj)); // [["a",1],["b",2]]

Object.seal(obj);
obj.a = 100;      // ☑ allowed (editing existing)
obj.c = 3;         // ✗ ignored (can't add new)

Object.freeze(obj);
obj.a = 999;        // ✗ ignored (fully locked)
```

Nested Object, Dot vs Bracket Notation, Optional Chaining

Optional chaining (?.) safely accesses a nested property and returns undefined instead of throwing an error if a step along the way doesn't exist — extremely useful when working with data that might be incomplete (like an API response).

```
let user = { profile: { name: "Alice" } };
console.log(user.profile?.name);   // "Alice"
console.log(user.address?.city);   // undefined, no crash
// Without optional chaining, the line above would throw a TypeError
```

Looping Objects — for...in, for...of, entries

for...in loops over an object's keys directly. for...of loops over the values of any iterable (arrays, strings, Maps) — it does not work directly on plain objects. Combine for...of with Object.entries() to get both key and value from an object in one loop.

```
for (let key in obj) {
  console.log(key, obj[key]);           // for...in: keys of an object
}

for (let value of [10, 20, 30]) {
  console.log(value);                     // for...of: values of an array
}

for (let [key, value] of Object.entries(obj)) {
  console.log(key, value);                // for...of + entries: both
}
```

Primitive vs Non-Primitive Data Types

Primitives (Number, String, Boolean, null, undefined, Symbol, BigInt) hold a single, immutable value directly, and are copied by value when assigned to a new variable. Non-primitives — objects, arrays, and functions — are reference types: a variable holds a pointer to the data in memory, not the data itself, so copying the variable copies the reference, not the underlying data.

```
let a = 5;
let b = a;    // b gets a COPY of the value 5
b = 10;
console.log(a); // still 5

let obj1 = { x: 1 };
let obj2 = obj1; // obj2 points to the SAME object
obj2.x = 100;
console.log(obj1.x); // 100 — obj1 changed too!
```

null vs undefined

undefined means a variable has been declared but never assigned a value — JavaScript sets it automatically. null is an intentional 'no value' that a developer explicitly assigns to signal 'this is empty on purpose.' typeof null is famously (and confusingly) "object", a long-standing quirk of the language.

```
let a;           // undefined — not yet assigned
let b = null;    // null — deliberately empty
console.log(typeof a); // "undefined"
console.log(typeof b); // "object" (quirk)
console.log(a == b);   // true  (loose equality treats them alike)
console.log(a === b);  // false (different types)
```

Truthy and Falsy Values in JavaScript

In a Boolean context (like an if condition), only these values are falsy: 0, "" (empty string), null, undefined, NaN, and false itself. Every other value is truthy — including "0" (a non-empty string), [] (empty array), and {} (empty object), which often surprises beginners.

```
if (0) console.log("runs");       // ❌ does NOT run (0 is falsy)
if ("") console.log("runs");      // ❌ does NOT run
if ("0") console.log("runs");     // ☑ DOES run (non-empty string)
if ([]) console.log("runs");       // ☑ DOES run (empty array is truthy)
if ({}) console.log("runs");        // ☑ DOES run (empty object is truthy)
```

== vs === and Implicit Conversion

== compares values after converting types if they differ (loose equality), which can cause surprising results like "" == 0 being true. === compares both value and type with no conversion, so it's almost always the safer, more predictable choice.

```
console.log(5 == "5");   // true  (converts "5" to number 5)
console.log(5 === "5");  // false (number vs string, no conversion)
console.log(null == undefined);  // true
console.log(null === undefined); // false
```

Block Scope, Global Scope, and Hoisting

A variable declared with let or const only exists inside the nearest {} block it's declared in (block scope). A variable declared outside any function/block is globally scoped and accessible everywhere. Hoisting means declarations are processed before code runs line by line, but let/const stay unusable in a 'temporal dead zone' until their actual declaration line executes — unlike var, which is hoisted and usable (as undefined) even before its declaration line.

(Advanced) Closure in Detail

A closure is a function that 'remembers' the variables from the scope it was created in, even after that outer function has already finished running. This is how you create private state in JavaScript — variables that can only be accessed or changed through the functions that closed over them.

```
function counter() {
  let count = 0;          // private variable, not accessible from outside
  return function () {
    count++;
    return count;
  };
}
const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2 — 'count' persisted between calls
```

(Optional) Callback Function and Passing Different Functions

A callback is a function passed as an argument to another function, to be called later — often after some operation completes, or as custom logic plugged into a more general function (like passing different callbacks to .map() or .filter()).

```
function processUser(name, callback) {
  let message = `Processing ${name}`;
  callback(message);
}
```

```
processUser("Alice", msg => console.log(msg));
processUser("Bob", msg => console.log(msg.toUpperCase()));
```

(Advanced) Pass by Reference vs Pass by Value

When a primitive is passed into a function, it's passed by value — a copy is made, so changes inside the function don't affect the original variable outside it. When an object or array is passed in, it's passed by reference — the function receives a pointer to the same data in memory, so mutating it (push, or changing a property) does affect the original.

```
function changeVal(x) { x = 100; }
let num = 5;
changeVal(num);
console.log(num); // still 5 — primitive, passed by value

function changeArr(arr) { arr.push(4); }
let nums = [1, 2, 3];
changeArr(nums);
console.log(nums); // [1, 2, 3, 4] — object/array, passed by reference
```

Pre/Post Increment and Pre/Post Decrement

x++ (post-increment) returns the CURRENT value first, then increases it afterward. ++x (pre-increment) increases the value FIRST, then returns the new value. The same logic applies to -- (decrement). The distinction only matters when the operator is used inline inside a larger expression.

```
let x = 5;
console.log(x++); // logs 5 (old value), then x becomes 6
console.log(x);    // 6
console.log(++x); // increases first → logs 7

let y = 5;
console.log(y--); // logs 5, then y becomes 4
console.log(--y); // decreases first → logs 3
```

Array Power Methods — map, forEach, filter, find

map() returns a brand-new array with every element transformed by a callback (same length as original). forEach() runs a function once per element but returns undefined — used purely for side effects like logging. filter() returns a new, possibly shorter array containing only elements that pass a test. find() returns just the first single element that passes a test (or undefined if none match).

```
let nums = [1, 2, 3, 4];

let doubled = nums.map(n => n * 2);        // [2,4,6,8]
```

```
nums.forEach(n => console.log(n));           // just logs each, returns
undefined
let evens = nums.filter(n => n % 2 === 0);   // [2,4]
let firstBig = nums.find(n => n > 2);         // 3 (first match only)
```

Array Power Methods — slice, reduce

slice(start, end) extracts a section of an array without changing the original (non-mutating). reduce() combines every element into a single accumulated value (a total, an object, anything), using a callback function and a starting value for the accumulator.

```
let nums = [1, 2, 3, 4];
let part = nums.slice(1, 3);              // [2,3] — original untouched
let total = nums.reduce((sum, n) => sum + n, 0); // 10

// reduce can build more than numbers — here, an object count:
let fruits = ["apple","banana","apple"];
let count = fruits.reduce((acc, f) => {
  acc[f] = (acc[f] || 0) + 1;
  return acc;
}, {});
console.log(count); // { apple: 2, banana: 1 }
```

Fixing the Order Status Bug & Receipt Generator

Practical debugging task: trace a status flag through conditionals to find where the logic breaks (often an == vs === mix-up, or a truthy/falsy trap), then use template strings to format a clean, multiline receipt from order data.

```
function generateReceipt(order) {
  return `Order #${order.id}
Customer: ${order.customer}
Total: $${order.total.toFixed(2)}`;
}
```

Default Parameters + Spread/Rest — Total Calculator & Array Merge

Rest parameters (...args) collect any number of arguments into a single array inside a function — the reverse of the spread operator, which expands an array into arguments.

```
function calculateTotal(...prices) {
  return prices.reduce((sum, p) => sum + p, 0);
}
calculateTotal(10, 20, 30); // 60

function mergeArrays(...arrays) {
  return [].concat(...arrays);
```

```
}
mergeArrays([1,2], [3,4], [5]); // [1,2,3,4,5]
```

Arrow Functions — Calculate Shipping and Get Grade

Rewriting small utility functions as arrow functions keeps calculation and grading logic short and readable, especially when used inline with array methods or as one-off callbacks.

```
const calculateShipping = weight => weight > 5 ? weight * 2 : 10;
const getGrade = score =>
  score >= 90 ? "A" : score >= 75 ? "B" : score >= 60 ? "C" : "F";
```

Object and Array Destructuring — API Response Extractor & Variable Swap

Destructuring is the standard way to pull specific fields out of a JSON/API response object without repeating response.field over and over, and it also lets you swap two variables' values without a temporary variable.

```
let a = 1, b = 2;
[a, b] = [b, a]; // swap in one line, no temp variable

const apiResponse = { id: 101, status: "ok", data: { name: "Alice" } };
const { id, status, data: { name } } = apiResponse; // nested destructuring
```

Object Methods and Optional Chaining — Most Expensive Product Finder

Combine reduce() with optional chaining to safely find the highest-priced item in an array of product objects, even if some entries have missing or nested fields, without the function crashing.

```
function mostExpensive(products) {
  return products.reduce((max, p) =>
    (p.price ?? 0) > (max.price ?? 0) ? p : max
  );
}
```

Map and Filter — Bulk Discount Applier and Passing Students Filter

Chain map() to apply a discount across every price in an array, and filter() to keep only students whose score meets a passing threshold — both return new arrays, leaving the originals unchanged.

```
let prices = [100, 200, 300];
let discounted = prices.map(p => p * 0.9); // 10% off everything

let passing = students.filter(s => s.score >= 60);
```

Find and Reduce — User Lookup and Shopping Cart Total

find() locates a single matching object (like a user by ID) instead of returning a whole filtered array. reduce() collapses cart items into a single total price by accumulating price × quantity across every item.

```
let user = users.find(u => u.id === 5);

let cartTotal = cart.reduce((sum, item) => sum + item.price * item.qty, 0);
```

Chaining Array Methods — Filtered Cart Total

Array methods can be chained directly since each one returns a new array — filter, then map, then reduce, all in a single readable pipeline, avoiding intermediate variables.

```
let total = cart
  .filter(item => item.inStock)
  .map(item => item.price * item.qty)
  .reduce((sum, val) => sum + val, 0);
```

Closures —

A practical closure exercise: build a function that keeps private, persistent state (like remaining assignment attempts) across multiple calls, without exposing that state to outside code directly — reinforcing the closure concept with a real scenario.

```
function createAssignment(maxAttempts) {
  let attemptsLeft = maxAttempts;
  return function attempt() {
    if (attemptsLeft <= 0) return "No attempts left";
    attemptsLeft--;
    return `Attempt used. ${attemptsLeft} remaining.`;
  };
}
const assignment = createAssignment(3);
console.log(assignment()); // "Attempt used. 2 remaining."
```

Callbacks and Reference Bugs — Sort and Mutation Fix

.sort() (like .reverse() and .splice()) mutates the original array in place, which can silently break code elsewhere that still expects the original order — a classic reference-type bug. Copy the array first with the spread operator [...arr] before sorting if the original must stay unchanged.

```
let original = [5, 3, 1];
let sorted = [...original].sort((a, b) => a - b); // copy first, then sort
console.log(sorted);   // [1, 3, 5]
console.log(original); // untouched: [5, 3, 1]
```